Chapter 5

Architecture Design

This chapter provides a detailed exploration of the system architecture, which is designed to be modular, scalable, and secure. Key components include the integration of LLMs, the use of RAG architecture, frontend and backend frameworks, and supporting services such as databases, CI/CD pipelines, and security layers. Each architectural component is designed to interact seamlessly while ensuring optimal performance and robustness.

5.1 Detailed Component Architecture

5.1.1 Backend Design

The backend architecture follows a microservices-based approach, ensuring that services are modular, scalable, and secure. Each microservice operates independently and can be scaled as needed based on demand.

At the core of this architecture is the **API Gateway**, which routes incoming client requests to the appropriate microservices. The system integrates with external services, including Single Sign-On (SSO) for user authentication and authorization.

Data Management: The backend efficiently manages data storage and retrieval, ensuring high responsiveness even under heavy loads. It also handles interactions with the LLMs and the RAG architecture, providing intelligent responses to user queries.

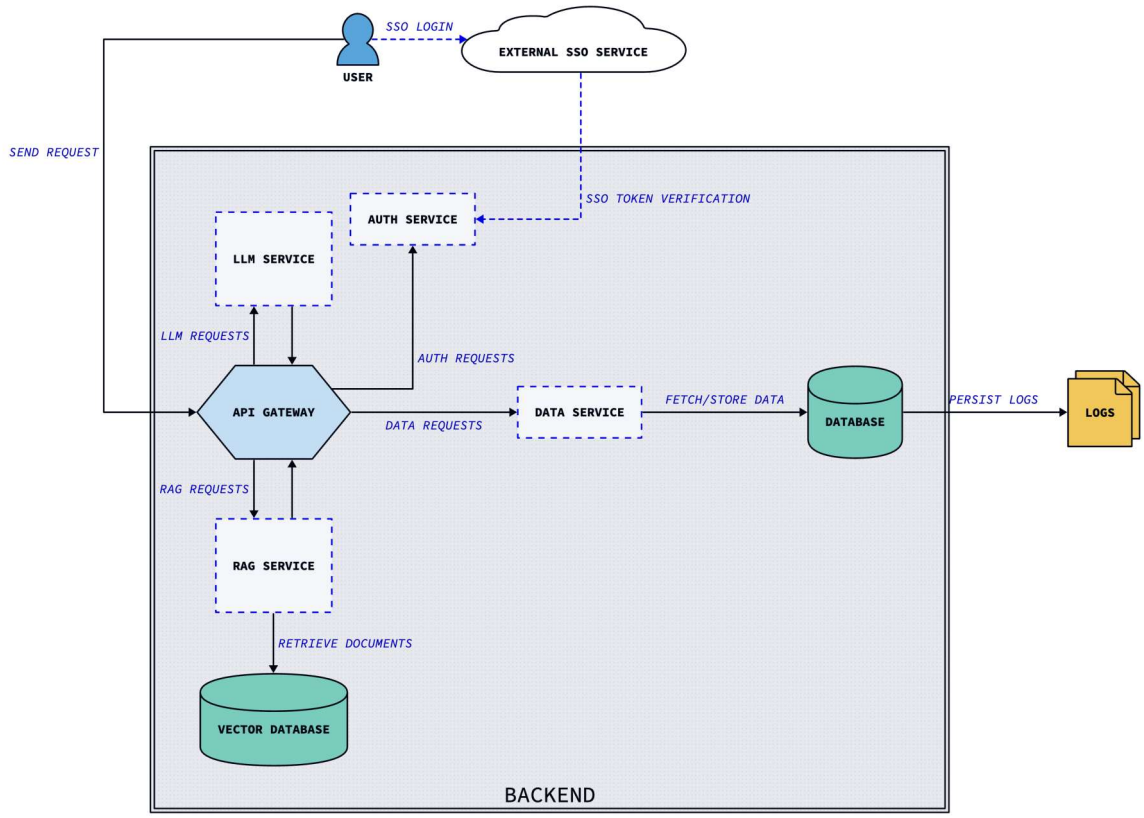

Figure 5.1: Backend Architecture Overview

5.1.2 LLM Integration

The integration of LLMs in the system leverages **LangChain**, which abstracts the specific LLM implementations and provides a unified interface for LLM communication. This LLM-agnostic approach allows the system to interact with different LLM technologies as they evolve, enhancing flexibility.

LangChain manages data flow between the system and the LLMs, optimizing API calls, handling input/output formatting, and implementing mechanisms for error handling and rate limiting. This design ensures that the system can efficiently adapt to new advancements in LLM technology without requiring significant architectural changes.

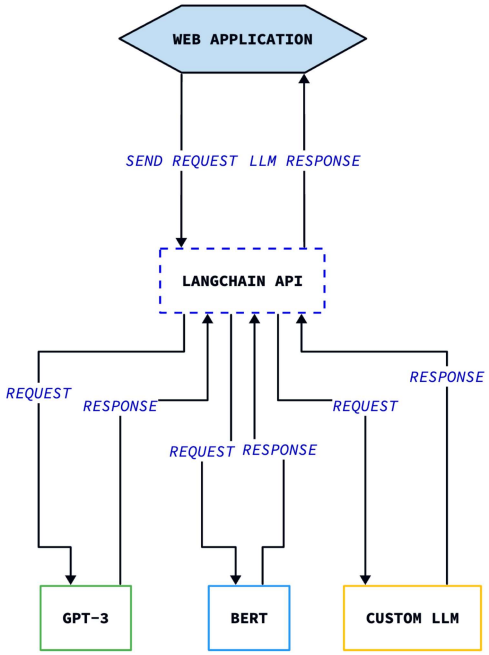

Figure 5.2: LLM Integration Architecture with LangChain

5.1.3 RAG Architecture

The **RAG** architecture combines retrieval and generation mechanisms to enhance the capabilities of LLMs. In this system, **Qdrant**, a high-performance vector database, is utilized for efficient similarity search and retrieval of relevant information from large datasets.

Query Preprocessing: The input query is preprocessed to improve retrieval accuracy. This includes normalizing the query by removing variations and focusing on key terms that enhance relevance. Once processed, the query is passed to Qdrant, which retrieves the most similar document chunks based on vector embeddings.

Reranking: Retrieved chunks are reranked based on their relevance to the query, ensuring the LLM

receives the most pertinent information. The top-ranked chunks, along with the processed query, are used for prompt construction.

Prompt Generation: The constructed prompt, including the document chunks and query, is fed into the LLM, which generates responses token by token. This approach ensures the generated content is contextually accurate and relevant.

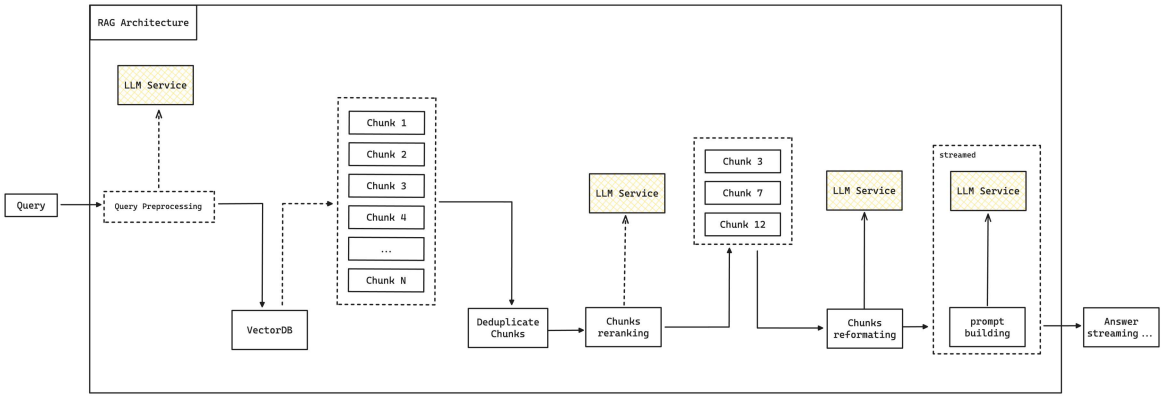

Figure 5.3: RAG Architecture with Query Preprocessing and Reranking

5.1.4 Frontend Design

The frontend architecture utilizes a **Single Page Application (SPA)** framework, implemented using either **React** or **Angular**, depending on project needs. For lightweight, fast-paced applications, **React** is preferred, while **Angular** is better suited for larger, more structured projects.

The frontend architecture consists of several key components:

- **SPA Framework:** Responsible for rendering the user interface and managing user interactions.
- **State Management:** Ensures consistent and predictable application state across components.
- **API Layer:** Manages communication between the frontend and backend, handling API requests and responses.
- **UI Components:** Dynamically rendered components based on the application state, providing a responsive user experience.

The frontend communicates with the backend through RESTful APIs, with an API Gateway handling requests. This design ensures modularity, scalability, and maintainability.

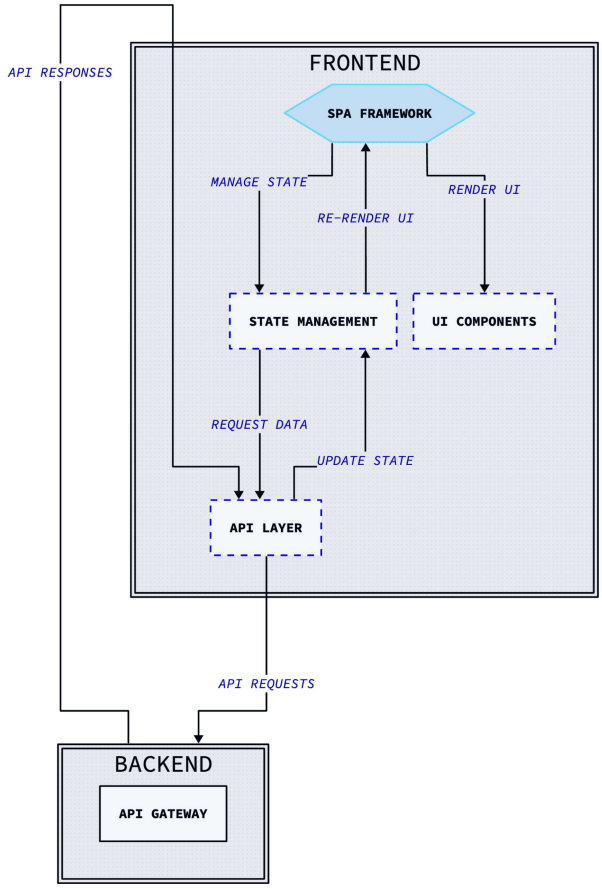

Figure 5.4: Frontend Architecture with SPA Framework and Backend Communication

5.1.5 Security Architecture

Security is a critical aspect of the architecture, given the sensitivity of the data being processed. The architecture incorporates comprehensive security measures, including authentication, authorization, data encryption, and secure communication.

Identity and Access Management: The system utilizes **Keycloak** for managing user identities and access control, ensuring that only authorized users can access specific resources. Single Sign-On (SSO) is employed to provide a unified login experience.

Data Encryption: All data, both in transit and at rest, is encrypted using industry-standard protocols. Secure communication is enforced through HTTPS, protecting the integrity and confidentiality of transmitted data.

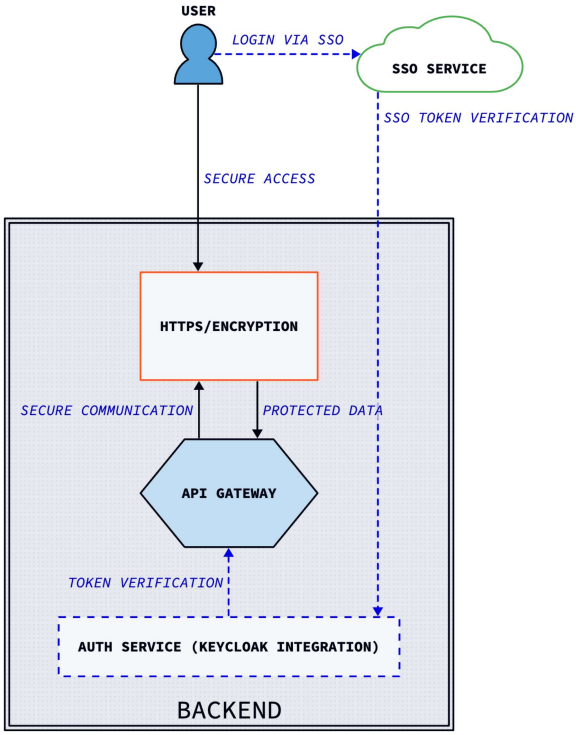

Figure 5.5: Security Integration with Keycloak and SSO

5.1.6 DevOps and Infrastructure

The system's infrastructure is designed for scalability, maintainability, and automated deployment using a **CI/CD pipeline**, containerization via **Docker**, and infrastructure as code with **Terraform**.

CI/CD Pipeline: The pipeline automates testing, building, and deployment processes, ensuring updates are smoothly integrated into production without downtime. The **testing and quality assurance** phase incorporates automated testing—including **unit tests**, **integration tests**, and **end-to-end tests** using tools like **Pytest**, **Jest**, and **Cypress**—with code coverage metrics monitored for effectiveness. In the **integration and deployment** phase, **continuous integration** automatically builds and tests code changes upon commit using **GitHub Actions**, while **continuous deployment/delivery** deploys applications to staging and production environments using Kubernetes deployment configurations.

Containerization and Orchestration: Docker is used for containerization, ensuring consistent environments across development, testing, and production. **Kubernetes** manages container orchestration, enabling the system to scale based on demand, perform rolling updates, and maintain desired state configurations.

Infrastructure as Code: Terraform is employed to manage infrastructure resources declaratively,

allowing version control and automated provisioning of cloud resources across providers like AWS, Azure, or Google Cloud.

Monitoring and Logging: Monitoring tools track system performance and health metrics, providing insights for proactive maintenance:

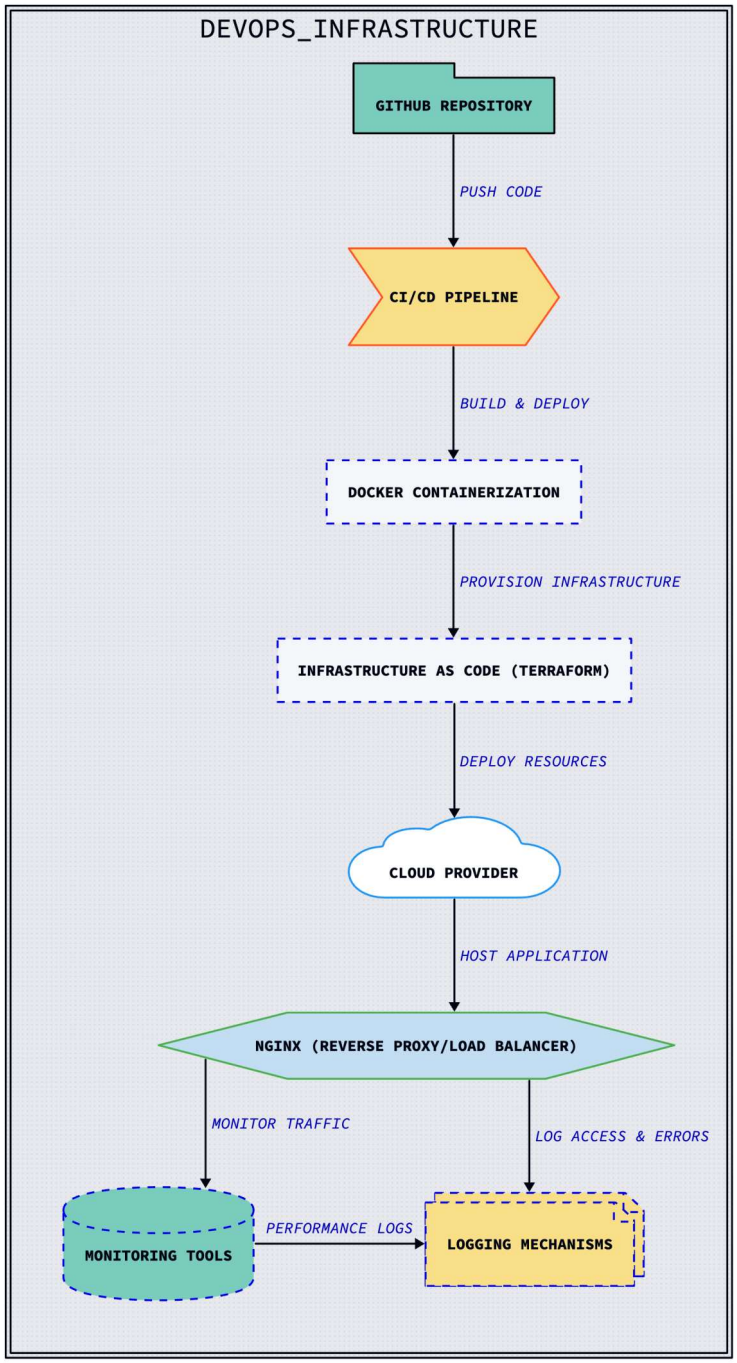

Figure 5.6: DevOps and Infrastructure Architecture showcasing CI/CD pipeline, containerization, and monitoring

5.1.7 Achieving 99.9% Availability

To meet the goal of 99.9% system availability, which translates to a maximum downtime of approximately 43.8 minutes per month, the following strategies are employed. Cloud providers such as AWS, Azure, and Google Cloud ensure that their infrastructure maintains availability above 99.9%, leveraging the mechanisms described below.

Redundancy and Fault Tolerance

Redundancy is implemented across all critical components, including considerations for external dependencies:

- **LLMs:** Recognizing the dependency on commercial LLMs such as OpenAI's GPT-4 and Google's Gemini, the system incorporates redundancy by integrating multiple LLM providers where possible. LangChain's abstraction allows for seamless switching between LLMs.
- **RAG System:** The vector database (e.g., Qdrant) uses active-active replication to ensure continuous access to the data for retrieval and generation.
- **Backend Services:** Backend microservices are replicated across multiple nodes and availability zones, ensuring that service failures in one node or zone do not affect overall system availability.
- **Databases:** Database systems employ replication strategies, such as master-slave or multi-master configurations, with automatic failover to standby instances.
- **Frontend Static Files:** The frontend is served via CDNs, which cache and distribute static assets globally. This reduces the load on origin servers and ensures high availability and low latency.

Load Balancing

Load balancers distribute incoming traffic across multiple service instances:

- **Backend Services:** Load balancers distribute API requests across multiple backend service instances, preventing any single instance from becoming a bottleneck.
- **Databases:** Read replicas are utilized to distribute database read operations, while write operations are managed to maintain data consistency.
- **Frontend Services:** CDNs and web servers employ load balancing to handle high volumes of requests for static assets.

Auto-scaling

Auto-scaling mechanisms dynamically adjust system resources:

- **Backend Microservices:** Kubernetes orchestrates horizontal pod autoscaling based on CPU usage, memory, or custom metrics.
- **Database Scaling:** Databases scale vertically or horizontally depending on demand, utilizing cloud provider features like Amazon RDS Read Replicas or Google Cloud SQL.
- **Frontend Scaling:** CDNs handle scaling for frontend assets, ensuring content is served efficiently to users worldwide.

Failover Mechanisms

Failover mechanisms ensure seamless operation during failures:

- **Service Failover:** Kubernetes automatically restarts failed containers and reschedules them on healthy nodes.
- **Database Failover:** Automatic promotion of standby databases to primary in the event of a failure.
- **Cross-Region Failover:** In case of a regional outage, services can failover to instances in other regions with minimal downtime.

Monitoring and Observability

Continuous monitoring is essential to maintain high availability:

- **Health Checks:** Regular health checks at application and infrastructure levels detect failures early.
- **Alerting Systems:** Automated alerts notify the operations team of issues before they impact users.
- **Performance Metrics:** Monitoring response times, error rates, and resource utilization helps in proactive scaling and troubleshooting.

Disaster Recovery

Disaster recovery plans are in place to handle catastrophic failures:

- **Regular Backups:** Automated backups of databases and critical data are stored in multiple regions.
- **Disaster Recovery Drills:** Regular testing of disaster recovery procedures ensures readiness.
- **Data Replication Across Regions:** Data is replicated across regions to enable quick recovery.

5.2 Conclusion

This chapter has provided a comprehensive overview of the system's architecture, designed with a focus on scalability, modularity, security, and high availability. Each architectural component—ranging from the backend microservices to the integration of LLMs using LangChain, the RAG architecture, the frontend Single Page Application (SPA) framework, and the underlying DevOps infrastructure—has been meticulously crafted to ensure robust and seamless interactions between services.

The system's backend architecture, built around microservices and an API gateway, offers modularity and scalability, while the LLM integration through LangChain ensures adaptability to evolving AI technologies. The RAG architecture, in conjunction with a vector database, enhances the system's ability to retrieve and generate relevant information efficiently, while the frontend architecture provides a responsive user experience.

Security measures, including authentication, authorization, encryption, and secure communication protocols, safeguard the system against threats, ensuring that data remains protected. Furthermore, the DevOps pipeline, containerization with Docker, orchestration with Kubernetes, and infrastructure management with Terraform allow for continuous delivery, automated scaling, and maintenance.

Achieving 99.9% availability is ensured through redundancy, fault tolerance, load balancing, auto-scaling, and robust monitoring mechanisms. These strategies, combined with disaster recovery plans, ensure that the system remains resilient even under high load or failure conditions.

Overall, the architecture detailed in this chapter is designed to support a high-performance, flexible, and secure system, capable of meeting the demands of modern applications.